FILE 1 — ALGORITHM ANALYSIS WITH PYTHON CODE

Lab 4: Smart Optimization Assistant using Dynamic Programming

Student Name: Deepika Rani

Roll No: 2401201131

Course / Section: BCA(AI & DS)-A

1. Introduction

Dynamic Programming (DP) solves a problem by dividing it into overlapping subproblems, solving each subproblem once, and storing the results for reuse. This lab implements three DP algorithms: Longest Common Subsequence (LCS), 0/1 Knapsack, and Matrix Chain Multiplication.

2. Longest Common Subsequence (LCS)

Aim: Given two sequences, find the length and one actual longest subsequence that appears in both sequences in the same relative order. Characters do not need to be consecutive.

Recurrence:

If X[i−1] == Y[j−1]: dp[i][j] = dp[i−1][j−1] + 1; otherwise dp[i][j] = max(dp[i−1][j], dp[i][j−1]).

Python Implementation (with comments):

```
def lcs(x, y):
    # m and n are the lengths of the two input strings
    m, n = len(x), len(y)

    # Create a DP table filled with zeros
    dp = [[0] * (n + 1) for _ in range(m + 1)]

    # Fill the table from smaller prefixes to larger prefixes
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if x[i - 1] == y[j - 1]:
                # Matching characters extend the previous LCS
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                # Keep the best result by skipping one character
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])

    # Trace backward to reconstruct one LCS string
    i, j = m, n
    answer = []
    while i > 0 and j > 0:
        if x[i - 1] == y[j - 1]:
            answer.append(x[i - 1])
            i -= 1
            j -= 1
        elif dp[i - 1][j] >= dp[i][j - 1]:
            i -= 1
        else:
            j -= 1
```

```
    answer.reverse()
    return dp[m][n], "".join(answer), dp


# Example execution
x = "ABCBDAB"
y = "BDCABA"
length, sequence, table = lcs(x, y)
print("LCS length:", length)
print("LCS:", sequence)
print("DP Table:")
for row in table:
    print(row)
```

Expected check: LCS length is 4. One valid LCS is BCBA; the implementation may return another valid LCS of the same length.

Case	Time Complexity	Space Complexity
All cases	O(m × n)	O(m × n)

3. 0/1 Knapsack

Aim: Given items with weights and values and a fixed capacity, select each item either completely or not at all to maximize total value without exceeding capacity.

Recurrence: If weight[i−1] ≤ w, dp[i][w] = max(dp[i−1][w], value[i−1] + dp[i−1][w−weight[i−1]]); otherwise dp[i][w] = dp[i−1][w].

Python Implementation (with comments):

```
def knapsack_01(weights, values, capacity):
    # Check that each item has both a weight and a value
    if len(weights) != len(values):
        raise ValueError("Weights and values must have equal length")

    n = len(weights)

    # dp[i][w] = best value using first i items and capacity w
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    # Build the table row by row
    for i in range(1, n + 1):
        item_weight = weights[i - 1]
        item_value = values[i - 1]

        for w in range(capacity + 1):
            # Option 1: do not select the current item
            dp[i][w] = dp[i - 1][w]

            # Option 2: select it, if it fits
            if item_weight <= w:
                dp[i][w] = max(
                    dp[i][w],
                    item_value + dp[i - 1][w - item_weight]
                )

    # Trace backward to find the selected items
    selected = []
    w = capacity
```

```
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(i - 1)  # zero-based item index
            w -= weights[i - 1]

    selected.reverse()
    return dp[n][capacity], selected, dp



# Example execution
weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
capacity = 5

best_value, selected_items, table = knapsack_01(
    weights, values, capacity
)
print("Maximum value:", best_value)
print("Selected item numbers:", [i + 1 for i in selected_items])
print("DP Table:")
for row in table:
    print(row)
```

Expected check: Maximum value is 7. The selected items are item 1 (weight 2, value 3) and item 2 (weight 3, value 4).

Case	Time Complexity	Space Complexity
All cases	O(n × W)	O(n × W)

Note: This is pseudo-polynomial time because runtime depends on the numeric capacity W as well as the number of items n.

4. Matrix Chain Multiplication

Aim: Given a sequence of matrices, find the parenthesization that minimizes the number of scalar multiplications. The algorithm finds the best order; it does not multiply the matrices.

Recurrence: cost[i][j] = min over i ≤ k < j of {cost[i][k] + cost[k+1][j] + p[i−1] × p[k] × p[j]}.

Python Implementation (with comments):

```
def matrix_chain_order(dims):
    # dims = [p0, p1, ..., pn] describes n matrices
    n = len(dims) - 1

    # cost[i][j] stores the minimum multiplication cost
    cost = [[0] * (n + 1) for _ in range(n + 1)]

    # split[i][j] stores the best place to split the chain
    split = [[0] * (n + 1) for _ in range(n + 1)]

    # chain_length is the number of matrices in the current chain
    for chain_length in range(2, n + 1):
        for i in range(1, n - chain_length + 2):
            j = i + chain_length - 1
            cost[i][j] = float("inf")

            # Try every possible split point k
            for k in range(i, j):
                current = (
                    cost[i][k]
```

```
                    + cost[k + 1][j]
                    + dims[i - 1] * dims[k] * dims[j]
                )

                # Save the minimum cost and its split point
                if current < cost[i][j]:
                    cost[i][j] = current
                    split[i][j] = k

    return cost[1][n], split, cost


def build_order(split, i, j):
    # A single matrix needs no further splitting
    if i == j:
        return f"A{i}"

    # Reconstruct the optimal parenthesization
    k = split[i][j]
    left = build_order(split, i, k)
    right = build_order(split, k + 1, j)
    return f"({left} x {right})"


# Example execution
dimensions = [10, 30, 5, 60]
minimum_cost, split_table, cost_table = matrix_chain_order(dimensions)
number_of_matrices = len(dimensions) - 1

print("Minimum scalar multiplications:", minimum_cost)
print("Optimal order:", build_order(
    split_table, 1, number_of_matrices
))
```

Expected check: For dimensions [10, 30, 5, 60], the minimum cost is 4500 scalar multiplications and the optimal order is ((A1 × A2) × A3).

Case	Time Complexity	Space Complexity
All cases	$O(n^3)$	$O(n^2)$

5. Overall Comparison

Algorithm	Purpose	Time	Space
LCS	Sequence comparison	O(m × n)	O(m × n)
0/1 Knapsack	Resource allocation	O(n × W)	O(n × W)
Matrix Chain	Computation-order optimization	$O(n^3)$	$O(n^2)$

6. Conclusion

The three implementations demonstrate bottom-up Dynamic Programming. LCS builds a table for pairs of sequence prefixes, Knapsack builds a table for item prefixes and capacities, and Matrix Chain Multiplication builds cost and split tables for matrix intervals. Each implementation returns the optimal result and exposes its DP table for visualization.